

SmartBugs - Previous Test Case Specification

Luciano Bercini

April 2026

Contents

1	Introduction	2
2	Functional Requirements of the Original System	2
3	Functional Testing of the Original SmartBugs Workflow	2
3.1	Test Frame	2
3.2	Test Frame Table	4
4	Test Case Specification	4
4.1	Test Case TC-PRE-01	4
4.2	Test Case TC-PRE-02	4
4.3	Test Case TC-PRE-03	5
4.4	Test Case TC-PRE-04	5
4.5	Test Case TC-PRE-05	5
4.6	Test Case TC-PRE-06	5
4.7	Test Case TC-PRE-07	6
4.8	Test Case TC-PRE-08	6
4.9	Test Case TC-PRE-09	6
4.10	Test Case TC-PRE-10	6
4.11	Test Case TC-PRE-11	7

1 Introduction

This document describes the pre-modification test case specification for the original SmartBugs system.

The goal of this document is to define a set of functional test cases that describe the expected behavior of SmartBugs before the proposed maintenance and evolution activities. These test cases are derived from the reverse engineering analysis of the system and from the existing pytest test suite.

Unlike projects where no test suite is available, SmartBugs already provides tests for several core components. Therefore, the existing tests are used as implementation-level evidence for the functional scenarios described in this document.

2 Functional Requirements of the Original System

After the reverse engineering phase, the following functional requirements were identified for the original SmartBugs workflow.

Requirement	Description
FR_1: File Collection	The system must be able to collect Solidity source files and bytecode files from user-provided file patterns.
FR_2: Tool Loading	The system must be able to load one or more analysis tools from their configuration files, including aliases and tools supporting multiple execution modes.
FR_3: Task Generation	The system must be able to generate analysis tasks by pairing collected files with compatible tools.
FR_4: Solidity Compiler Handling	The system must be able to resolve the Solidity compiler version required by tools that depend on <code>solc</code> .
FR_5: Main Workflow Orchestration	The system must orchestrate the complete workflow by loading tools, collecting files, generating tasks, and delegating execution to the analysis module.

Table 1: Original SmartBugs Functional Requirements

The following sections focus on the functional testing of these requirements before the proposed modifications. Although the original SmartBugs workflow can be described through multiple functional requirements, the testing activity focuses mainly on FR_5, since it represents the complete orchestration workflow. The remaining requirements are tested as functional steps involved in the execution of FR_5.

3 Functional Testing of the Original SmartBugs Workflow

The original SmartBugs workflow is based on command-line configuration. The user provides input files, selects one or more analysis tools, and the system creates the corresponding analysis tasks. These tasks are then executed using the existing SmartBugs analysis pipeline.

Based on the reverse engineering analysis, the main parameters involved in the original workflow are:

- **inputFile**: the input contract or bytecode file to analyze;
- **filePattern**: the glob pattern used to collect files;
- **toolConfiguration**: the selected analysis tool or set of tools;
- **executionMode**: the file/tool compatibility mode, such as Solidity, bytecode, or runtime bytecode;
- **settings**: execution settings such as runtime mode, result directory, and error handling behavior.

3.1 Test Frame

Description

The system must correctly collect input files, load tools, generate compatible analysis tasks, and orchestrate the original SmartBugs workflow.

Parameters

- `inputFile`

- filePattern
- toolConfiguration
- executionMode
- settings

Categories

Category Name	Category Values
inputFile	IF1: valid Solidity file IF2: valid bytecode file IF3: runtime bytecode file IF4: file without Solidity pragma IF5: non-existing file
filePattern	FP1: single file pattern FP2: glob pattern matching multiple files FP3: recursive glob pattern FP4: empty pattern list
toolConfiguration	TC1: valid single tool TC2: valid multiple tools TC3: tool alias TC4: duplicate tool identifiers TC5: tool requiring <code>solc</code>
executionMode	EM1: Solidity mode EM2: bytecode mode EM3: runtime bytecode mode EM4: incompatible file/tool mode
settings	S1: standard settings S2: runtime flag enabled S3: continue-on-errors enabled S4: result directory collision possible

Table 2: Category Partition for the Original SmartBugs Workflow

Constraints

- IF1 can be paired with tools supporting Solidity mode.
- IF2 can be paired with tools supporting bytecode mode.
- IF3 can be paired with tools supporting runtime mode.
- IF4 produces an error if the selected tool requires `solc`.
- IF5 must not generate analysis tasks.
- EM4 must not produce a task.
- Duplicate files must not produce duplicate tasks.
- Tool aliases must be expanded into the corresponding concrete tools.

3.2 Test Frame Table

Test Case ID	Test Frame	Expected Result
TC-PRE-01	IF1, FP1	A single Solidity file is collected correctly.
TC-PRE-02	IF1, FP2	Multiple Solidity files matching a glob pattern are collected.
TC-PRE-03	IF2, FP1, EM2	A bytecode file is collected and matched with a bytecode tool.
TC-PRE-04	IF3, FP1, EM3	A runtime bytecode file is collected and matched with a runtime tool.
TC-PRE-05	IF1, TC1, EM1	One task is generated for a valid Solidity file and a compatible tool.
TC-PRE-06	IF1, TC2, EM1	Multiple tasks are generated for one file and multiple compatible tools.
TC-PRE-07	IF4, TC5	An error is raised because no pragma is available for <code>solc</code> resolution.
TC-PRE-08	IF1, EM4	No task is generated when the tool mode is incompatible with the input file.
TC-PRE-09	TC3	A tool alias is correctly expanded into its concrete tools.
TC-PRE-10	S4	Result directory collisions are resolved by assigning unique directories.
TC-PRE-11	S1	The main workflow correctly invokes tool loading, file collection, task generation, and analysis execution.

Table 3: Test Frames for the Original SmartBugs Workflow

4 Test Case Specification

4.1 Test Case TC-PRE-01

Test Case ID	TC-PRE-01
Test Frame	IF1, FP1
Requirement	FR_1
Scenario	Collection of a single valid Solidity file.
Pre-conditions	SmartBugs is installed and a valid Solidity file named <code>Test.sol</code> exists.
Input	A file pattern pointing directly to <code>Test.sol</code> .
Expected Result	The system collects exactly one file, returns its absolute path, and recognizes it as a valid Solidity input.
Existing Test Coverage	<code>test_collect_single_sol_file</code>

Table 4: Test Case TC-PRE-01

4.2 Test Case TC-PRE-02

Test Case ID	TC-PRE-02
Test Frame	IF1, FP2
Requirement	FR_1
Scenario	Collection of multiple Solidity files using a glob pattern.
Pre-conditions	SmartBugs is installed and a directory contains multiple <code>.sol</code> files and at least one non-Solidity file.
Input	A glob pattern such as <code>*.sol</code> .
Expected Result	The system collects only the Solidity files matching the pattern and ignores unrelated files.
Existing Test Coverage	<code>test_collect_multiple_sol_files_with_glob</code>

Table 5: Test Case TC-PRE-02

4.3 Test Case TC-PRE-03

Test Case ID	TC-PRE-03
Test Frame	IF2, FP1, EM2
Requirement	FR_1, FR_3
Scenario	Collection and task generation for a bytecode file.
Pre-conditions	A valid <code>.hex</code> file exists and a tool supporting bytecode mode is selected.
Input	A bytecode file and a compatible bytecode analysis tool.
Expected Result	The system creates one analysis task in bytecode mode.
Existing Test Coverage	<code>test_collect_hex_files</code> , <code>test_collect_tasks_hex_bytecode</code>

Table 6: Test Case TC-PRE-03

4.4 Test Case TC-PRE-04

Test Case ID	TC-PRE-04
Test Frame	IF3, FP1, EM3
Requirement	FR_3
Scenario	Task generation for runtime bytecode.
Pre-conditions	A valid <code>.rt.hex</code> file exists and a runtime-compatible tool is selected.
Input	A runtime bytecode file and a runtime analysis tool.
Expected Result	The system creates one task in runtime mode.
Existing Test Coverage	<code>test_collect_tasks_runtime_bytecode</code>

Table 7: Test Case TC-PRE-04

4.5 Test Case TC-PRE-05

Test Case ID	TC-PRE-05
Test Frame	IF1, TC1, EM1
Requirement	FR_3
Scenario	Basic task generation for a Solidity file and a compatible tool.
Pre-conditions	A valid Solidity file exists, settings are frozen, and a compatible Solidity tool is selected.
Input	One Solidity file and one Solidity-mode tool.
Expected Result	The system generates exactly one task containing the correct file path, relative path, and tool identifier.
Existing Test Coverage	<code>test_collect_tasks_basic</code>

Table 8: Test Case TC-PRE-05

4.6 Test Case TC-PRE-06

Test Case ID	TC-PRE-06
Test Frame	IF1, TC2, EM1
Requirement	FR_3
Scenario	Task generation for one Solidity file and multiple compatible tools.
Pre-conditions	A valid Solidity file exists and multiple compatible Solidity tools are selected.
Input	One Solidity file and two Solidity-mode tools.
Expected Result	The system generates one task for each compatible tool.
Existing Test Coverage	<code>test_collect_tasks_multiple_tools</code>

Table 9: Test Case TC-PRE-06

4.7 Test Case TC-PRE-07

Test Case ID	TC-PRE-07
Test Frame	IF4, TC5
Requirement	FR_4
Scenario	Task generation for a Solidity file without pragma when the selected tool requires <code>solc</code> .
Pre-conditions	A Solidity file without pragma exists and a tool requiring Solidity compiler resolution is selected.
Input	A Solidity file without pragma and a <code>solc</code> -dependent tool.
Expected Result	The system raises an error because the compiler version cannot be determined.
Existing Test Coverage	<code>test_collect_tasks_no_pragma</code>

Table 10: Test Case TC-PRE-07

4.8 Test Case TC-PRE-08

Test Case ID	TC-PRE-08
Test Frame	IF1, EM4
Requirement	FR_3
Scenario	Incompatible file and tool mode.
Pre-conditions	A valid Solidity file exists and a tool supporting only bytecode mode is selected.
Input	A <code>.sol</code> file and a bytecode-only tool.
Expected Result	The system does not generate any task because the tool mode is incompatible with the input file type.
Existing Test Coverage	<code>test_collect_tasks_mode_mismatch</code>

Table 11: Test Case TC-PRE-08

4.9 Test Case TC-PRE-09

Test Case ID	TC-PRE-09
Test Frame	TC3
Requirement	FR_2
Scenario	Loading a tool alias.
Pre-conditions	A tool alias is defined and points to two concrete tools.
Input	A tool identifier representing an alias.
Expected Result	The system expands the alias and loads the concrete tools referenced by it.
Existing Test Coverage	<code>test_load_tool_alias</code>

Table 12: Test Case TC-PRE-09

4.10 Test Case TC-PRE-10

Test Case ID	TC-PRE-10
Test Frame	S4
Requirement	FR_3
Scenario	Handling result directory collisions.
Pre-conditions	Two tasks are generated with a result directory template that may produce the same path.
Input	One Solidity file, two compatible tools, and a result directory template that may collide.
Expected Result	The system assigns unique result directories to the generated tasks.
Existing Test Coverage	<code>test_collect_tasks_result_dir_collisions</code>

Table 13: Test Case TC-PRE-10

4.11 Test Case TC-PRE-11

Test Case ID	TC-PRE-11
Test Frame	S1
Requirement	FR_5
Scenario	Main SmartBugs orchestration workflow.
Pre-conditions	SmartBugs settings are initialized and one analysis tool is selected.
Input	A valid settings object containing selected tools and input files.
Expected Result	The system freezes settings, loads tools, collects files, generates tasks, and invokes the analysis module.
Existing Test Coverage	<code>test_main_orchestration_flow</code> , <code>test_main_freezes_settings</code>

Table 14: Test Case TC-PRE-11